

1.	Pre-requisite	2
2.	Java API – Producer – JSON Object Serialization- 60 Minutes	3
3.	Java API – Consumer – Json Deserialzation - 60 Minutes	28
4.	Schema registry – 90 Minutes	41
5.	Errors	60
1.	LEADER_NOT_AVAILABLE	60
	java.util.concurrent.ExecutionException:	60
6.	Annexure Code:	63
2.	DumplogSegment	63
3.	Data Generator – JSON.....	64
7.	Pom.xml (Standalone).....	72

Last Updated: 17 September 2026

JDK : Jdk : 1.17

Apache kafka : kafka_2.13-3.9.0.tar

1. Pre-requisite

Create a Java Maven Project and replace the pom.xml content from the Annexure.

Maven java project Details:

Group ID: com.tos.kafka

Artifact ID: LearningKafkaZK

Maven Build and eclipse source build should be 1.17 or above.

You need to start zookeeper and kafka broker node/s before going ahead.

```
[root@tos scripts]# jps
4880 Kafka
4881 Kafka
4882 Kafka
6022 Jps
4845 QuorumPeerMain
[root@tos scripts]#
```

Here, as shown above three Kafka broker services and ZK service need to be started. You can have one Kafka node also.

Install confluent Kafka
Registry and confluent CLI

2. Java API – Producer – JSON Object Serialization- 60 Minutes

Learning Outcomes:

- JSON Messages Serialization
- Async and Sync Mode.
- Callback method

In this lab, we will be sending a JSON object using a custom serialize and in async mode. The Stock Price Producer example has the following classes:

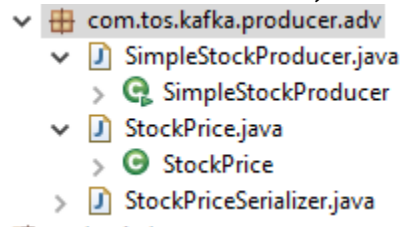
- StockPrice - holds a stock price has a name, dollar, and cents
- SimpleStockProducer - Configures and creates KafkaProducer, StockSender list, ThreadPool (ExecutorService), starts StockSender runnable into thread pool
- StockPriceSerializer - can serialize a StockPrice into byte[]

You can use the same maven project we have created in the producer example. In order to categorize the tutorials, let us create a package `com.tos.kafka.aproducer`, all the classes created for this lab will be stored inside this package.

Package Name:

com.tos.kafka.producer.adv

At the end of this lab, we will have the following class.



StockPrice

The StockPrice is a simple domain object that holds a stock price has a name, dollar, and cents. The StockPrice knows how to convert itself into a JSON string.

We are using Gson library to convert into json.

```
package com.tos.kafka.producer.adv;
```

```
import com.google.gson.Gson;
```

```
public class StockPrice {  
  
    private final int dollars;  
    private final int cents;
```

```

private final String name;
static Gson jfactory = new Gson();
public StockPrice(final String json) {

    this(jfactory.fromJson( json, StockPrice.class));
}

public StockPrice() {
    dollars = 0;
    cents = 0;
    name = "";
}

public StockPrice(final String name, final int dollars, final
int cents) {
    this.dollars = dollars;
    this.cents = cents;
    this.name = name;
}

public StockPrice(final StockPrice stockPrice) {
    this.cents = stockPrice.cents;
    this.dollars = stockPrice.dollars;
    this.name = stockPrice.name;
}

```

```
}
```

```
public int getDollars() {  
    return dollars;  
}
```

```
public int getCents() {  
    return cents;  
}
```

```
public String getName() {  
    return name;  
}
```

```
@Override  
public String toString() {  
    return "StockPrice{" +  
        "dollars=" + dollars +  
        ", cents=" + cents +  
        ", name='" + name + '\'' +  
        '}';  
}
```

```
}
```

```
@Override
```

```
public boolean equals(Object o) {
```

```
    if (this == o) return true;
```

```
    if (o == null || getClass() != o.getClass()) return false;
```

```
    StockPrice that = (StockPrice) o;
```

```
    if (dollars != that.dollars) return false;
```

```
    if (cents != that.cents) return false;
```

```
    return name != null ? name.equals(that.name) : that.name ==
```

```
    null;
```

```
}
```

```
@Override
```

```
public int hashCode() {
```

```
    int result = dollars;
```

```
    result = 31 * result + cents;
```

```
    result = 31 * result + (name != null ? name.hashCode() :
```

```
    0);
```

```
    return result;
```

```
}
```

```

    public String toJson() {
        return "{" +
            "\"dollars\": " + dollars +
            ", \"cents\": " + cents +
            ", \"name\": \"" + name + "\"" +
            "}";
    }
}

```

Custom Serializers

We are not using built-in Kafka serializers. We are writing your own custom serializer. You just need to be able to convert your custom keys and values using the serializer convert to and convert from byte arrays (byte[]). Serializers work for keys and values, and you set them up with the Kafka Producer properties value.serializer, and key.serializer. The StockPriceSerializer will serialize StockPrice into bytes.

Create the following Custom Serializer class

package com.tos.kafka.producer.adv;

import java.nio.charset.StandardCharsets;

import java.util.Map;

```
import org.apache.kafka.common.serialization.Serializer;
```

```
public class StockPriceSerializer implements Serializer<StockPrice> {
```

```
    @Override
```

```
    public byte[] serialize(String topic, StockPrice data) {
        return data.toJson().getBytes(StandardCharsets.UTF_8);
    }
```

```
    @Override
```

```
    public void configure(Map<String, ?> configs, boolean isKey) {
    }
```

```
    @Override
```

```
    public void close() {
    }
}
```

You need to override the serialize method to convert JSON to bytes so that it can stream the stock price json document to broker.

SimpleStockProducer

Create a class `SimpleStockProducer`, which will be our main program that will send records to Broker. Here we will demonstrate pushing of JSON document using serialize component. `SimpleStockProducer` import classes and sets up a logger. It has a `main()` method to create a `KafkaProducer` instance. It has a `initProducer()` method to initialize bootstrap servers, client id, key serializer and custom serializer (`StockPriceSerializer`). It has a `main()` method that creates the producer, creates a `StockSender` list passing each instance the producer, and then it runs each `stockSender` in its own thread.

```
package com.tos.kafka.producer.adv;
```

```
public class SimpleStockProducer
```

Add the following import in the above class.

```
import java.util.Properties;  
import java.util.concurrent.CountDownLatch;  
import java.util.concurrent.ExecutionException;  
import java.util.concurrent.TimeUnit;  
  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.Producer;  
import org.apache.kafka.clients.producer.ProducerRecord;  
import org.apache.kafka.clients.producer.RecordMetadata;
```

Add the following method to initialize the properties of the Producer.

```
public final static String TOPIC = "stock-prices";

public static Properties initProducer() {
    // Assign topicName to string variable

    // create instance for properties to access producer
    configs
    Properties props = new Properties();

    // Assign localhost id
    props.put("bootstrap.servers", "kafka0:9092");
    // props.put("bootstrap.servers", "localhost:9092");

    // Set acknowledgements for producer requests.
    props.put("acks", "all");
    props.put("enable.auto.commit", "true");
    // If the request fails, the producer can automatically
    retry,
    props.put("retries", 1);

    // Specify buffer size in config
    props.put("batch.size", 1);
```

```

        // Reduce the no of requests less than 0
        props.put("linger.ms", 100);

        // The buffer.memory controls the total amount of memory
        // available to the
        // producer for buffering.
        props.put("buffer.memory", 302);

        props.put("key.serializer",
"org.apache.kafka.common.serialization.StringSerializer");

        props.put("value.serializer",
StockPriceSerializer.class.getName());
        // props.put("transactional.id", "my-transactional-id");
        return props;
    }

```

The main method that will create the producer and send message to Broker

```

public static void main(String[] args) {

    // invokeSync ()

```

```

        // invokeAsync ()

    }

    public static void invokeSync () {
        Properties props = initProducer();
        Producer<String, StockPrice> producer = new KafkaProducer<String,
StockPrice>(props);
        sendMessage(producer, TOPIC);
    }

    public static void invokeAsync () {
        Properties props = initProducer();
        Producer<String, StockPrice> producer = new KafkaProducer<String,
StockPrice>(props);

        sendMessageAsync(producer, TOPIC);

    }

```

Add the following function in the main class to send records synchronously to Kafka Producer.

```

public static void sendMessage(Producer<String, StockPrice> producer, String topic) {

    for (int i = 0; i < 2; i++) {
        try {
            ProducerRecord <String, StockPrice> record =
createRandomRecord(i);
            RecordMetadata future = producer.send(record).get();
            System.out.println(">>>" + future.topic() + " Offset:" +
future.offset());
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        } catch (ExecutionException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        producer.flush();
        System.out.println("Message sent successfully");
    }
    producer.close();
    // test(producer);
    System.out.println("Message sent successfully & Close");
}

```

Method that will create Two Producer records using StockPrice class

```

private static ProducerRecord<String, StockPrice> createRandomRecord(int i) {
    final int dollarAmount = i * 200;
    final int centAmount = i * 1000;
    final StockPrice stockPrice = new StockPrice(" STOCK"+ i, dollarAmount,
centAmount);
    return new ProducerRecord<>(TOPIC, stockPrice.getName(), stockPrice);
}

```

To send records asynchronously with Kafka Producer the following method needs to be added.

Kafka provides an asynchronous send method to send a record to a topic. Let's use this method to send some message ids and messages to the Kafka topic we created earlier. The big difference here will be that we use a lambda expression to define a callback.

// Sending Message Asynchronously

```

public static void sendMessageAsync(Producer<String, StockPrice> producer,
String topic) {
    final CountdownLatch countDownLatch = new CountdownLatch(2);
    for (int i = 0; i < 2; i++) {
        try {
            ProducerRecord <String, StockPrice> record =
createRandomAsyncRecord(i);

            long time = System.currentTimeMillis();

```

```

        // Register a call back.
        producer.send(record, (metadata, exception) -> {
            long elapsedTime = System.currentTimeMillis() - time;
            if (metadata != null) {
                System.out.printf("sent record(key=%s value=%s) " +
                    "meta(partition=%d, offset=%d) time=%d\n",
                    record.key(), record.value(), metadata.partition(),
                    metadata.offset(), elapsedTime);
            } else {
                exception.printStackTrace();
            }
            countDownLatch.countDown();
        });

        countDownLatch.await(25, TimeUnit.SECONDS);

    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    producer.flush();
    System.out.println("Message sent successfully");

```

```

    }
    producer.close();
    // test(producer);
    System.out.println("Message sent successfully & Close");
}

```

Notice the use of a `CountDownLatch` so we can send all N messages and then wait for them all to send.

Add the following code that will create Record for sending to broker.

```

private static ProducerRecord<String, StockPrice> createRandomAsyncRecord(int i) {
    final int dollarAmount = i * 200;
    final int centAmount = i * 1000;
    final StockPrice stockPrice = new StockPrice(" Async STOCK " + i,
dollarAmount, centAmount);
    return new ProducerRecord<>(TOPIC, stockPrice.getName(), stockPrice);
}

```

Async Interface Callback and Async Send Method

Kafka defines a [Callback](#) interface that you use for asynchronous operations. The callback interface allows code to execute when the request is complete. The callback executes in a

background I/O thread so it should be fast (don't block it).

The `onCompletion(RecordMetadata metadata, Exception exception)` gets called when the asynchronous operation completes. The `metadata` gets set (not null) if the operation was a success, and the exception gets set (not null) if the operation had an error.

The async `send` method is used to send a record to a topic, and the provided callback gets called when the `send` is acknowledged. The `send` method is asynchronous, and when called returns immediately once the record gets stored in the buffer of records waiting to post to the Kafka broker. The `send` method allows sending many records in parallel without blocking to wait for the response after each one.

Since the send call is asynchronous it returns a Future for the RecordMetadata that will be assigned to this record. Invoking `get()` on this future will block until the associated request completes and then return the metadata for the record or throw any exception that occurred while sending the record. `KafkaProducer`

In the above code, the following line overrides the `onCompletion(RecordMetadata metadata, Exception exception)` method.

```
producer.send(record, (metadata, exception)
```

On Kafka broker

Create the required topic

```
#/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --replication-factor 1 --partitions 1 --topic stock-prices
```

Open a terminal and execute the following command to consume the message:

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic stock-prices
```

```
^CProcessed a total of 0 messages
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic stock-prices --from-beginning
{"dollars": 0, "cents": 0, "name": " STOCK0"}
{"dollars": 200, "cents": 1000, "name": " STOCK1"}

```

The output will come once you execute the java program.

Execute the program one at a time by commenting and uncommenting the other methods in the main method().

a) `invokeSync();`

```
138  
139 public static void main(String[] args) {  
140  
141     invokeSync();  
142     // invokeAsync();  
143  
144 }  
145
```

Result

```
>>>stock-prices Offset:0  
Message sent successfully  
>>>stock-prices Offset:1  
Message sent successfully  
Message sent successfully & Close
```

Now invoke the async and comment the earlier method.

b) `invokeAsync()`;

```

137         ,
138
139         public static void main(String[] args) {
140
141             // invokeSync();
142             | invokeAsync();
143
144     }
```

Result:

```
sent record(key= Async STOCK 0 value=StockPrice{dollars=0, cents=0,
name=' Async STOCK 0'}) meta(partition=0, offset=2) time=582
```

Message sent successfully

```
sent record(key= Async STOCK 1 value=StockPrice{dollars=200,
cents=1000, name=' Async STOCK 1'}) meta(partition=0, offset=3)
time=114
```

Message sent successfully

```
sent record(key= Async STOCK 2 value=StockPrice{dollars=400,
cents=2000, name=' Async STOCK 2'}) meta(partition=0, offset=4)
time=9
Message sent successfully
sent record(key= Async STOCK 3 value=StockPrice{dollars=600,
cents=3000, name=' Async STOCK 3'}) meta(partition=0, offset=5)
time=8
Message sent successfully
Message sent successfully & Close
```

On the consumer console.

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic stock-prices --from-beginning
{"dollars": 0, "cents": 0, "name": " STOCK0"}
{"dollars": 200, "cents": 1000, "name": " STOCK1"}
{"dollars": 0, "cents": 0, "name": " Async STOCK 0"}
{"dollars": 200, "cents": 1000, "name": " Async STOCK 1"}
{"dollars": 400, "cents": 2000, "name": " Async STOCK 2"}
{"dollars": 600, "cents": 3000, "name": " Async STOCK 3"}
█
```

Let us verify the distribution of message in a partition topic too.

Create a topic with 4 partitions as shown below:

```
# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --replication-factor 1 --partitions 4 --topic mystock-prices
```

Update the topic name as shown below in `SimpleStockProducer.java` file.

```
--  
14  
15 public class SimpleStockProducer {  
16  
17     public final static String TOPIC = "mystock-prices";  
18  
19 public static Properties initProducer() {
```

We have created the topic, **mystock-prices** with 4 partitions so that message will be distributed across the partition.

Ensure that the following method is not commented.

```

SimpleStockProducer.java  StockPrice.java  StockPriceSerializer.java
133         final int dollarAmount = i* 200;
134         final int centAmount = i * 1000;
135         final StockPrice stockPrice = new StockPrice(" Async STOCK " + i);
136         return new ProducerRecord<>(TOPIC, stockPrice.getName(), stockPrice.serialize());
137     }
138
139     public static void main(String[] args) {
140
141         //invokeSync();
142         invokeAsync();
143     }
144 }
145

```

We will send 4 messages as shown, updated the counter to 4.

```
// Sending Message Asynchronously
public static void sendMessageAsync(Producer<String, StockPrice> producer, String
    final CountdownLatch countDownLatch = new CountdownLatch(2);
    for (int i = 0; i < 4; i++) {
        try {
            ProducerRecord<String, StockPrice> record = createRandomAsyncRecord

            long time = System.currentTimeMillis();
            // Register a call back.
            producer.send(record, (metadata, exception) -> {
                long elapsedTime = System.currentTimeMillis() - time;
                if (metadata != null) {
                    System.out.printf("sent record(key=%s value=%s) "
                        "meta(partition=%d, offset=%d) tim
                        record.key(), record.value(), metadata.par
                        metadata.offset(), elapsedTime);
                } else {
                    exception.printStackTrace();
                }
            })
        }
    }
}
```

Execute the main program.

As you can see from the console, there are distributed across different partition.

```
<terminated> SimpleStockProducer [Java Application] D:\Apps\Java\jdk1.8.0_171\bin\javaw.exe (26-Jun-2018, 4:58:35 PM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
sent record(key= Async STOCK 0 value=StockPrice{dollars=0, cents=0, name=' Async STOCK 0'}) meta(partition=2, offset=0) time=340
Message sent successfully
sent record(key= Async STOCK 1 value=StockPrice{dollars=200, cents=1000, name=' Async STOCK 1'}) meta(partition=3, offset=0) time=158
Message sent successfully
sent record(key= Async STOCK 2 value=StockPrice{dollars=400, cents=2000, name=' Async STOCK 2'}) meta(partition=2, offset=1) time=10
Message sent successfully
sent record(key= Async STOCK 3 value=StockPrice{dollars=600, cents=3000, name=' Async STOCK 3'}) meta(partition=1, offset=0) time=20
Message sent successfully
Message sent successfully & Close
```

You can describe the partition as shown below:

```
# /opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092 --topic mystock-prices
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092 --topic mystock-prices
Topic: mystock-prices   TopicId: 7BJPC0zcSN-zSxmA6dahzw PartitionCount: 4      ReplicationFactor: 1    Configs: segment.bytes=1073741824
    Topic: mystock-prices Partition: 0    Leader: 0      Replicas: 0    Isr: 0
    Topic: mystock-prices Partition: 1    Leader: 0      Replicas: 0    Isr: 0
    Topic: mystock-prices Partition: 2    Leader: 0      Replicas: 0    Isr: 0
    Topic: mystock-prices Partition: 3    Leader: 0      Replicas: 0    Isr: 0
[root@kafka0 scripts]#
```

Consumer:

```
# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic mystock-prices --from-beginning
```

```
Processed a total of 4 messages
[root@kafka0 scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic mystock-prices --from-beginning
{"dollars": 0, "cents": 0, "name": " Async STOCK 0"}
{"dollars": 200, "cents": 1000, "name": " Async STOCK 1"}
{"dollars": 400, "cents": 2000, "name": " Async STOCK 2"}
{"dollars": 600, "cents": 3000, "name": " Async STOCK 3"}

```

You can verify the messages in the partition log segment.

```
/opt/kafka/bin/kafka-run-class.sh kafka.tools.DumpLogSegments --deep-iteration --print-data-log --files \
/opt/data/kafka-logs/mystock-prices-0/00000000000000000000.log | head -n 4
```

```
[root@kafka0 /]# /opt/kafka/bin/kafka-run-class.sh kafka.tools.DumpLogSegments --deep-iteration --print-data-log --files /opt/data/kafka-logs/mystock-prices-1/00000000000000000000.log | head -n 4
Dumping /opt/data/kafka-logs/mystock-prices-1/00000000000000000000.log
Starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: -1 lastSequence: -1 producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1679211925721 size: 140 magic: 2 compresscodec: none crc: 1788891645 isValid: true
| offset: 0 CreateTime: 1679211925721 keySize: 14 valueSize: 57 sequence: -1 headerKeys: [] key: Async STOCK 3 payload: {"dollars": 600, "cents": 3000, "name": " Async STOCK 3"}
[root@kafka0 /]#
```

Replace the appropriate partition.

Lab Ends here

3. Java API – Consumer – Json Deserialization - 60 Minutes

Learning outcome:

- JSON deserialization and retry option

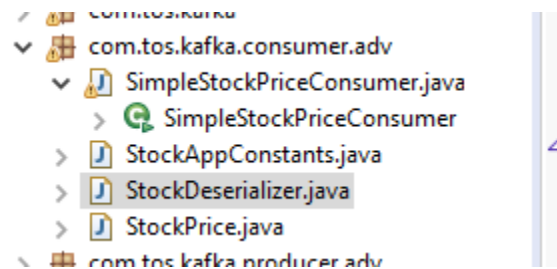
We will use the same Maven project that was created earlier.

Let us create a separate package for storing all the classes that will be created in this tutorial.

Package : `com.tos.kafka.consumer.adv`

We will demonstrate how to use JSON deserialization and retry option.

At the end of the lab, you should have the project structure as shown below:



The Stock Price Consumer example has the following classes:

- StockPrice - holds a stock price has a name, dollar, and cents
- SimpleStockPriceConsumer - consumes StockPrices and display batch lengths for poll
- StockAppConstants - holds topic and broker list
- StockDeserializer - can deserialize a StockPrice from byte[]

StockDeserializer

The StockDeserializer just calls the JSON parser to parse JSON in bytes to a StockPrice object. Create the following class.

```
package com.tos.kafka.consumer.adv;
```

```
import org.apache.kafka.common.serialization.Deserializer;
```

```
import java.nio.charset.StandardCharsets;
```

```
import java.util.Map;
```

```
public class StockDeserializer implements Deserializer<StockPrice> {
```

```
    @Override
```

```
    public StockPrice deserialize(final String topic, final byte[] data) {  
        return new StockPrice(new String(data, StandardCharsets.UTF_8));
```

```

    }

    @Override
    public void configure(Map<String, ?> configs, boolean isKey) {
    }

    @Override
    public void close() {
    }
}

```

We just need to override the `deserialize()` that convert the JSON bytes to Stock Price. Create the following class which will be our data or domain value object.

```

package com.tos.kafka.consumer.adv;

import com.google.gson.Gson;

public class StockPrice {

    private final int dollars ;
    private final int cents ;
    private final String name ;
    static Gson gson = new Gson();
}

```

```
public StockPrice(final String json) {  
    this(gson.fromJson(json, StockPrice.class));  
}
```

```
public StockPrice() {  
    dollars = 0;  
    cents = 0;  
    name = "";  
}
```

```
public StockPrice(final String name, final int dollars, final  
int cents) {  
    this.dollars = dollars;  
    this.cents = cents;  
    this.name = name;  
}
```

```
public StockPrice(final StockPrice stockPrice) {  
    this.cents = stockPrice.cents;  
    this.dollars = stockPrice.dollars;  
    this.name = stockPrice.name;  
}
```

```
public int getDollars() {  
    return dollars;  
}
```

```
public int getCents() {  
    return cents;  
}
```

```
public String getName() {  
    return name;  
}
```

```
@Override  
public String toString() {  
    return "StockPrice{" +  
        "dollars=" + dollars +  
        ", cents=" + cents +  
        ", name='" + name + '\'' +  
        '}';  
}
```

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;

    StockPrice that = (StockPrice) o;

    if (dollars != that.dollars) return false;
    if (cents != that.cents) return false;
    return name != null ? name.equals(that.name) : that.name ==
null;
}

@Override
public int hashCode() {
    int result = dollars;
    result = 31 * result + cents;
    result = 31 * result + (name != null ? name.hashCode() :
0);
    return result;
}

```

```

    public String toJson() {
        return "{" +
            "\"dollars\": " + dollars +
            ", \"cents\": " + cents +
            ", \"name\": \"" + name + "\"" +
            "}";
    }
}

```

The application constants that define the servers that we will be connecting by our consumer. Ensure that you change your system broker IP accordingly. Update the code with that of the following.

```

package com.tos.kafka.consumer.adv;

public class StockAppConstants {
    public final static String TOPIC = "stock-prices";
    public final static String BOOTSTRAP_SERVERS =
        "kafka0:9092";
}

```

SimpleStockPriceConsumer uses createConsumer method to create a KafkaConsumer instance, subscribes to stock-prices topics and has a custom deserializer.

It has a runConsumer() method that drains topic, creates List of current stocks and calls displayRecordsStatsAndStocks() method.

The method displayRecordsStatsAndStocks() prints out size of each partition read and total record count and prints out each stock at its current price.

Create a class **SimpleStockPriceConsumer** and import the following class.

```
package com.tos.kafka.consumer.adv;
```

```
import java.time.Duration;  
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.List;  
import java.util.Properties;  
import java.util.UUID;
```

```
import org.apache.kafka.clients.consumer.Consumer;  
import org.apache.kafka.clients.consumer.ConsumerConfig;  
import org.apache.kafka.clients.consumer.ConsumerRecords;  
import org.apache.kafka.clients.consumer.KafkaConsumer;  
import org.apache.kafka.common.serialization.StringDeserializer;
```

```
public class SimpleStockPriceConsumer
```

Add the following method that will create KafkaConsumer.

```
private static Consumer<String, StockPrice> createConsumer() {
    final Properties props = new Properties();
    props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
StockAppConstants.BOOTSTRAP_SERVERS);
    // props.put(ConsumerConfig.GROUP_ID_CONFIG,
    "KafkaExampleAdvConsumer");
    props.put(ConsumerConfig.GROUP_ID_CONFIG, "KA" +
UUID.randomUUID().toString());
    props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
"earliest");
    // props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
    // "KafkaExampleAdvConsumer");
    props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
    props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StockDeserializer.class.getName());
    props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);
    // Create the consumer using props.
    final Consumer<String, StockPrice> consumer = new
KafkaConsumer<>(props);
```

```
// Subscribe to the topic.
```

```
consumer.subscribe(Collections.singletonList(StockAppConstants.  
TOPIC));  
return consumer;  
}
```

Add the following method that will call the poll method that will pull the message from the broker

```
static void runConsumer() throws InterruptedException {  
    final Consumer<String, StockPrice> consumer =  
createConsumer();  
    final List<StockPrice> map = new ArrayList<StockPrice>();  
    try {  
        final int giveUp = 10;  
        int noRecordsCount = 0;  
  
        while (true) {  
            ConsumerRecords<String, StockPrice> consumerRecords  
= consumer.poll(Duration.ofMillis(2000));  
            if (consumerRecords.count() == 0) {  
                noRecordsCount++;  
                if (noRecordsCount > giveUp)  
                    break;  
            }  
        }  
    }  
}
```

```
        else  
            continue;  
    }  
    consumerRecords.forEach(record -> {  
        map.add(record.value());  
    });  
    displayRecordsStatsAndStocks(map, consumerRecords);  
    consumer.commitAsync();  
}  
} finally {  
    consumer.close();  
}  
System.out.println("DONE");  
}
```

The following method will print the records fetch from the Topic.

```

private static void displayRecordsStatsAndStocks(final List<StockPrice>
stockPriceMap,
final ConsumerRecords<String, StockPrice> consumerRecords) {
    System.out.printf("New ConsumerRecords partition count: %d Message count:
%d\n",
        consumerRecords.partitions().size(), consumerRecords.count());
    stockPriceMap.forEach((stockPrice) -> System.out.printf("ticker %s price
%d.%d \n", stockPrice.getName(),
        stockPrice.getDollars(), stockPrice.getCents()));
    System.out.println();
}

```

Finally update the following method to call the method that consume the record.

```

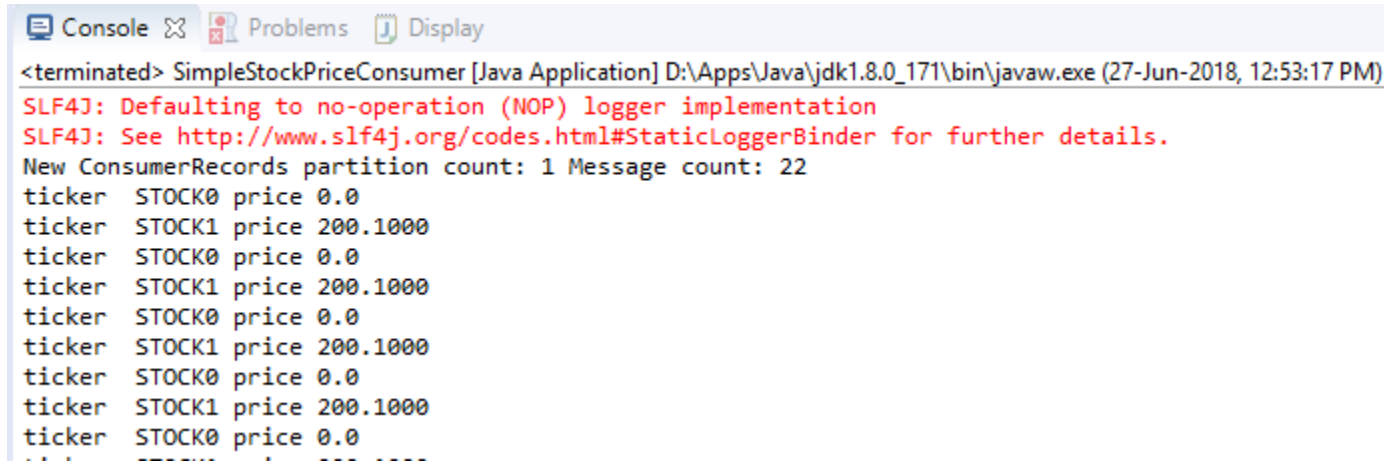
public static void main(String[] args) {
    try {
        runConsumer();
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

```

```
}
```

Ensure that you have started the broker before going ahead.

Execute the main program.



```
<terminated> SimpleStockPriceConsumer [Java Application] D:\Apps\Java\jdk1.8.0_171\bin\javaw.exe (27-Jun-2018, 12:53:17 PM)
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
New ConsumerRecords partition count: 1 Message count: 22
ticker STOCK0 price 0.0
ticker STOCK1 price 200.1000
ticker STOCK0 price 0.0
ticker STOCK1 price 200.1000
ticker STOCK0 price 0.0
ticker STOCK1 price 200.1000
ticker STOCK0 price 0.0
ticker STOCK1 price 200.1000
ticker STOCK0 price 0.0
.....
```

As you can see above, it printed the number of partition count and the number of messages too along with the messages. It may vary depending on the number of messages that was pushed by your producer.

-----Lab Ends here -----

4. Schema registry – 90 Minutes

Create a java maven project or use the earlier maven project[Preferable].

This tutorial provides a step-by-step workflow for using Confluent Schema Registry.

You will learn how to enable client applications to read and write Avro data, which has integrated capabilities with Schema Registry.

Create the transactions topic

For the exercises in this tutorial, you will be producing to and consuming from a topic called `transactions`.

using CLI:

```
#kafka-topics --create --bootstrap-server kafka0:9092 --partitions 1 --replication-factor 1 --  
topic transactions
```

Schema Definition

The first thing developers need to do is agree on a basic schema for data. Client applications form a contract:

- producers will write data in a schema
- consumers will be able to read that data

Consider the original Payment schema.

Copy the following content in ***src/main/resources/avro*** folder with file Name:***Payment.avsc***

```
{  
  "namespace": "com.tos.registry",  
  "type": "record",  
  "name": "Payment",  
  "fields": [  
    {"name": "id", "type": "string"},  
    {"name": "amount", "type": "double"}  
  ]  
}
```

Here is a break-down of what this schema defines:

- **namespace**: a fully qualified name that avoids schema naming conflicts
- **type**: Avro data type, for example, **record**, **enum**, **union**, **array**, **map**, or **fixed**
- **name**: unique schema name in this namespace
- **fields**: one or more simple or complex data types for a **record**. The first field in this record is called *id*, and it is of type *string*. The second field in this record is called *amount*, and it is of type *double*.

Project structure will be as shown after performing this lab.

Create corresponding package :

com.tos.registry

Update pom.xml if not done.

- Dependencies **org.apache.avro.avro** and **io.confluent.kafka-avro-serializer** to serialize data as Avro
- Plugin **avro-maven-plugin** to generate Java class files from the source schema

```
<plugin>
  <groupId>org.apache.avro</groupId>
  <artifactId>avro-maven-plugin</artifactId>
  <version>${avro.version}</version>
  <executions>
    <execution>
      <phase>generate-sources</phase>
      <goals>
```

```

        <goal>schema</goal>
      </goals>
      <configuration>
        <sourceDirectory>${project.basedir}/src/main/resources/avro</sourceDirectory>
        <includes>
          <include>*.avsc</include>
        </includes>
        <outputDirectory>${project.basedir}/src/main/java</outputDirectory>
      </configuration>
    </execution>
  </executions>
</plugin>

```

Configuring Avro

Kafka applications using Avro data and Schema Registry need to specify at least two configuration parameters:

- Avro serializer or deserializer
- Properties to connect to Schema Registry

There are two basic types of Avro records that your application can use:

- a specific code-generated class, or
- a generic record

The examples in this tutorial demonstrate how to use the specific **Payment** class. Using a specific code-generated class requires you to define and compile a Java class for your schema, but it is easier to work with in your code.

Java Producers

Within the application, Java producers need to configure the Avro serializer for the Apache Kafka® value (or Kafka key) and URL to Schema Registry. Then the producer can write records where the Kafka value is of `Payment` class.

Example Producer Code

When constructing the producer, configure the message value class to use the application's code-generated `Payment` class. For example:

```
// Code Begins Here
package com.tos.registry;

import java.util.Properties;

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.errors.SerializationException;
import org.apache.kafka.common.serialization.StringSerializer;

import io.confluent.kafka.serializers.AbstractKafkaSchemaSerDeConfig;
```

```

import io.confluent.kafka.serializers.KafkaAvroSerializer;
public class ProducerExample {

    private static final String TOPIC = "transactions";

    @SuppressWarnings("InfiniteLoopStatement")

    public static void main(final String[] args) {

        final Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
" kafka0:9092");
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.RETRIES_CONFIG, 0);
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class);
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
KafkaAvroSerializer.class);

        props.put(AbstractKafkaSchemaSerDeConfig.SCHEMA_REGISTRY_URL_CO
NFIG, "http://kafka0:8081");

        try (KafkaProducer<String, Payment> producer = new
KafkaProducer<String, Payment>(props)) {

```

```

        for (long i = 0; i < 10; i++) {
            final String orderId = "id" + Long.toString(i);
            final Payment payment = new Payment(orderId,
1000.00d);
            final ProducerRecord<String, Payment> record = new
ProducerRecord<String, Payment>(TOPIC,
                                payment.getId().toString(), payment);

            producer.send(record);

            Thread.sleep(1000L);
        }

        producer.flush();
        System.out.printf("Successfully produced 10 messages to
a topic called %s\n", TOPIC);

    } catch (final SerializationException e) {
        e.printStackTrace();
    } catch (final InterruptedException e) {
        e.printStackTrace();
    }
}

```

```
}
// Code Ends Here
```

Execute Maven → Generate Sources.

It will generate **Payment.java file** in the same package as the main class. It's specified in the pom.xml

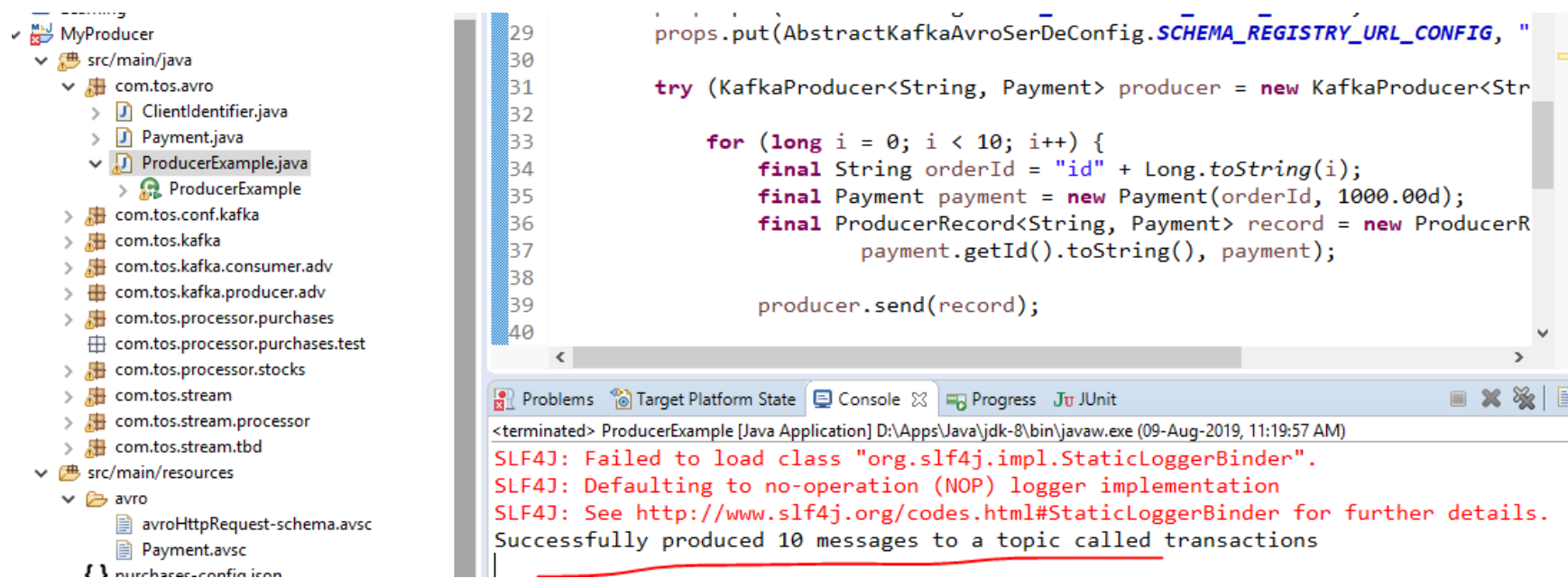
Pom.xml has the plugins for generating sources for avro. Ensure that avro schema file is in the below directories as specified below.

```
1  <plugin>
2      <groupId>org.apache.avro</groupId>
3      <artifactId>avro-maven-plugin</artifactId>
4      <version>${avro.version}</version>
5      <executions>
6          <execution>
7              <phase>generate-sources</phase>
8              <goals>
9                  <goal>schema</goal>
10             </goals>
11             <configuration>
12                 <sourceDirectory>${project.basedir}/src/main/resources/avro</sourceDirectory>
13                 <includes>
14                     <include>*.avsc</include>
15                 </includes>
16                 <outputDirectory>${project.basedir}/src/main/java</outputDirectory>
17             </configuration>
18         </execution>
19     </executions>
20 </plugin>
```

Start the schema registry along with the kafka broker.

Run the Producer

1. Run **ProducerExample**, which produces Avro-formatted messages to the **transactions** topic.



The screenshot shows an IDE with a project named 'MyProducer'. The file explorer on the left shows the project structure, including 'src/main/java' and 'src/main/resources'. The 'src/main/java' folder contains several packages, including 'com.tos.avro', 'com.tos.conf.kafka', 'com.tos.kafka', 'com.tos.kafka.consumer.adv', 'com.tos.kafka.producer.adv', 'com.tos.processor.purchases', 'com.tos.processor.purchases.test', 'com.tos.processor.stocks', 'com.tos.stream', 'com.tos.stream.processor', and 'com.tos.stream.tbd'. The 'com.tos.avro' package contains 'ClientIdentifier.java', 'Payment.java', and 'ProducerExample.java'. The 'com.tos.conf.kafka' package contains 'ProducerExample'.

The main editor shows the code for 'ProducerExample.java'. The code is as follows:

```

29 props.put(AbstractKafkaAvroSerDeConfig.SCHEMA_REGISTRY_URL_CONFIG, "
30
31 try (KafkaProducer<String, Payment> producer = new KafkaProducer<Str
32
33     for (long i = 0; i < 10; i++) {
34         final String orderId = "id" + Long.toString(i);
35         final Payment payment = new Payment(orderId, 1000.00d);
36         final ProducerRecord<String, Payment> record = new ProducerR
37             payment.getId().toString(), payment);
38
39         producer.send(record);
40

```

The console output at the bottom shows the following messages:

```

<terminated> ProducerExample [Java Application] D:\Apps\Java\jdk-8\bin\javaw.exe (09-Aug-2019, 11:19:57 AM)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Successfully produced 10 messages to a topic called transactions

```

You can verify the log in Registry.

[2023-03-20 13:10:18,710] INFO Registering new schema: subject transactions-value, version null, id null, type null, schema size 140

(io.confluent.kafka.schemaregistry.rest.resources.SubjectVersionsResource:323)

[2023-03-20 13:10:18,755] INFO Wait to catch up until the offset at 1

(io.confluent.kafka.schemaregistry.storage.KafkaStore:316)

[2023-03-20 13:10:18,755] INFO Reached offset at 1

(io.confluent.kafka.schemaregistry.storage.KafkaStore:318)

[2023-03-20 13:10:18,766] INFO Wait to catch up until the offset at 2

(io.confluent.kafka.schemaregistry.storage.KafkaStore:316)

[2023-03-20 13:10:18,784] INFO Reached offset at 2

(io.confluent.kafka.schemaregistry.storage.KafkaStore:318)

[2023-03-20 13:10:18,808] INFO 172.18.0.1 - - [20/Mar/2023:07:40:18 +0000] "POST /subjects/transactions-value/versions HTTP/1.1" 200 8 "-" "Java/11.0.9" 204

(io.confluent.rest-utils.requests:62)

ent.rest-utils.requests:62)

[2022-10-18 15:12:06,061] INFO Registering new schema: subject transactions-value, version null, id null, type null, schema size 140 (io.confluent.kafka.schemaregistry.rest.resources.SubjectVersionsResource:323)

[2022-10-18 15:12:06,062] INFO Wait to catch up until the offset at 15 (io.confluent.kafka.schemaregistry.storage.KafkaStore:316)

[2022-10-18 15:12:06,063] INFO Reached offset at 15 (io.confluent.kafka.schemaregistry.storage.KafkaStore:318)

[2022-10-18 15:12:06,069] INFO Wait to catch up until the offset at 16 (io.confluent.kafka.schemaregistry.storage.KafkaStore:316)

[2022-10-18 15:12:06,071] INFO Reached offset at 16 (io.confluent.kafka.schemaregistry.storage.KafkaStore:318)

The schema Id will be assigned and register in the Registry when producer is executed for the first time.

Verify the schema being created on the Registry using REST API Using CLI:

List the schema created on the Registry. Depending on the respository state, you may have some extra schemas created. However, you should get a schema with an id “**transactions-value**” here, **transactions** is the topic name.

```
# curl -X GET http://kafkao:8081/subjects
```

You can get more details of the schema by specifying the Schema ID as shown below

```
# curl -X GET http://kafkao:8081/subjects/transactions-value/versions/latest
```

```
[root@kafka0 kafka-logs]# curl -X GET http://localhost:8081/subjects
["transactions-value"]
[root@kafka0 kafka-logs]# curl -X GET http://localhost:8081/subjects/transactions-value/versions/latest
{"subject":"transactions-value","version":1,"id":1,"schema":{"type":"record","name":"Payment","namespace":"com.tos.registry","fields":[{"name":"id","type":"string"}, {"name":"amount","type":"double"}]}}
```

Using Consumer CLI, verify the messages:

```
# kafka-console-consumer --bootstrap-server kafka0:9092 --topic transactions --from-beginning
```

A terminal window with a black background and white text. The prompt is [root@kafka0 kafka-logs]#. The command executed is kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic transactions --from-beginning. The output shows a list of IDs from id0 to id9, each followed by a small icon of a person. The list is: id0@, id1@, id2@, id3@, id4@, id5@, id6@, id7@, id8@, id9@. A cursor is visible at the end of the last line.

```
Processed a total of 0 messages
[root@kafka0 kafka-logs]# kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic transactions --from-beginning
id0@
id1@
id2@
id3@
id4@
id5@
id6@
id7@
id8@
id9@
```

Or using Avroconsumer console.

```
#kafka-avro-console-consumer --topic transactions --bootstrap-server kafka0:9092 --from-beginning
```

```
[root@kafka0 /]# kafka-avro-console-consumer --topic transactions --bootstrap-server kafka0:9092 --from-beginning
{"id":"id0","amount":1000.0}
{"id":"id1","amount":1000.0}
{"id":"id2","amount":1000.0}
{"id":"id3","amount":1000.0}
{"id":"id4","amount":1000.0}
{"id":"id5","amount":1000.0}
{"id":"id6","amount":1000.0}
{"id":"id7","amount":1000.0}
{"id":"id8","amount":1000.0}
{"id":"id9","amount":1000.0}
```

Java Consumers

Let us consume the messages using Java API.

Within the client application, Java consumers need to configure the Avro deserializer for the Kafka value (or Kafka key) and URL to Schema Registry. Then the consumer can read records where the Kafka value is of `Payment` class.

Example Consumer Code

By default, each record is deserialized into an Avro `GenericRecord`, but in this tutorial the record should be deserialized using the application's code-generated `Payment` class.

Therefore, configure the deserializer to use Avro `SpecificRecord`, i.e., `SPECIFIC_AVRO_READER_CONFIG` should be set to `true`. For example:

```
// Code Begins Here
```

```
package com.tos.registry;
```

```
import java.time.Duration;  
import java.util.Collections;  
import java.util.Properties;
```

```
import org.apache.kafka.clients.consumer.ConsumerConfig;  
import org.apache.kafka.clients.consumer.ConsumerRecord;
```

```
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;

import io.confluent.kafka.serializers.AbstractKafkaAvroSerDeConfig;
import io.confluent.kafka.serializers.KafkaAvroDeserializer;
import io.confluent.kafka.serializers.KafkaAvroDeserializerConfig;
public class ConsumerExample {
    private static final String TOPIC = "transactions";

    @SuppressWarnings("InfiniteLoopStatement")
    public static void main(final String[] args) {
        final Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
" kafka0:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "test-payments");
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG,
" true");
        props.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG,
" 1000");
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
" earliest");

        props.put(AbstractKafkaAvroSerDeConfig.SCHEMA_REGISTRY_URL_CONFIG,
"http://kafka0:8081");
```

```

        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
KafkaAvroDeserializer.class);

props.put(KafkaAvroDeserializerConfig.SPECIFIC_AVRO_READER_CONFIG,
true);

        try (final KafkaConsumer<String, Payment> consumer = new
KafkaConsumer<>(props)) {

            consumer.subscribe(Collections.singletonList(TOPIC));
            while (true) {
                final ConsumerRecords<String, Payment> records =
consumer.poll(Duration.ofMillis(100));
                for (final ConsumerRecord<String, Payment> record :
records) {
                    //record.headers().forEach( p ->
System.out.print(p.key()));
                    final String key = record.key();
                    final Payment value = record.value();
                    System.out.printf("key = %s, value = %s%n",
key, value);

                }
            }
        }

```

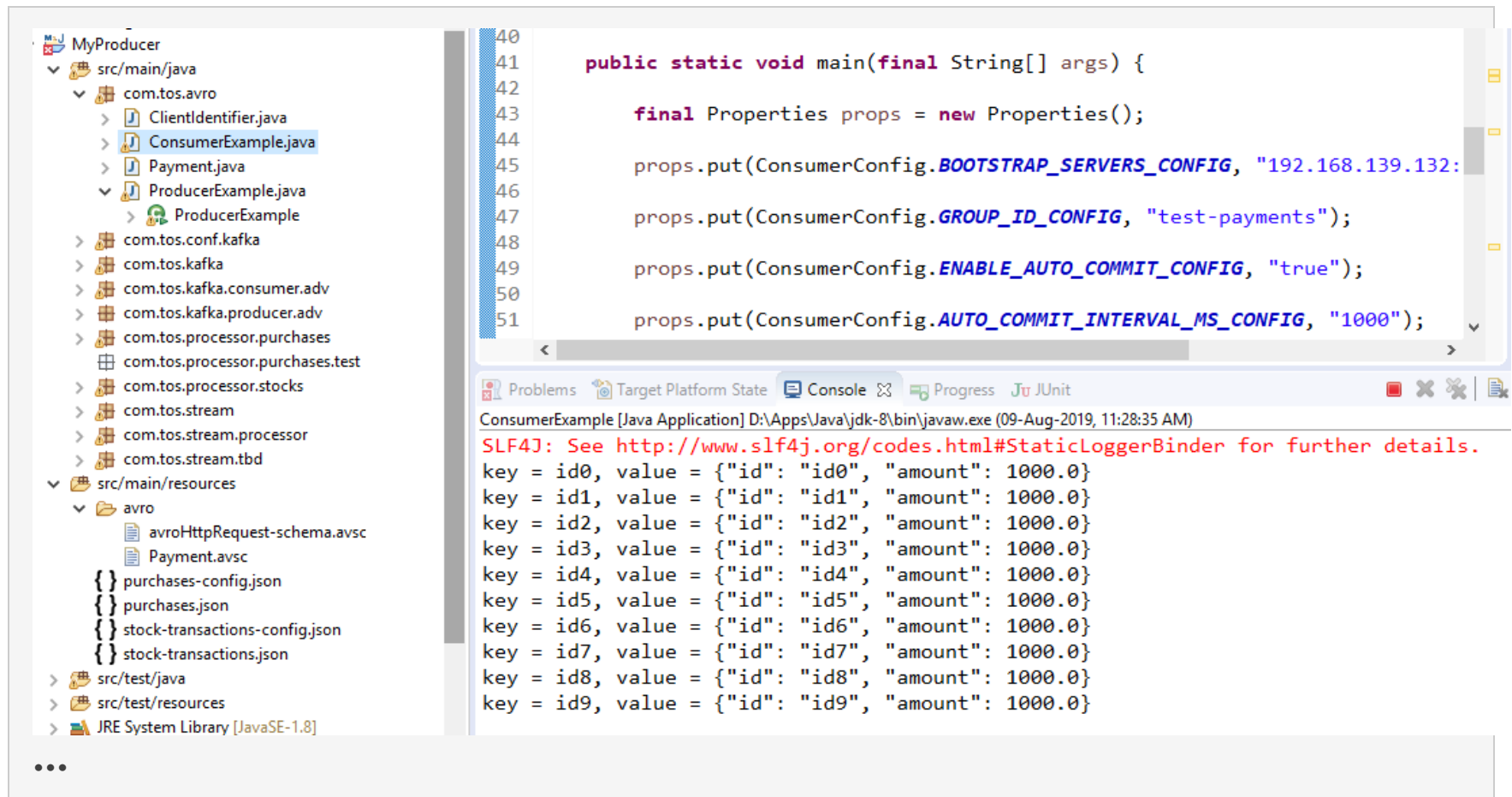
```
    }  
    }  
}
```

// Code Ends Here

Run the Consumer

1. run `ConsumerExample` (assuming you already ran the `ProducerExample` above).

You should see:



2. Hit **Ctrl-C** to stop.

You can verify the Schema registry Log.

```
[2022-10-18 15:46:07,252] INFO 172.18.0.1 - - [18/Oct/2022:10:16:07 +0000] "GET /schemas/ids/4 HTTP/1.1" 200 183 "-" "Java/11.0.9" 4 (io.confluent.rest-utils.requests:62)
```

It fetches the Detail of schema using the Ids.

----- Lab Ends Here -----

5. Errors

1. LEADER_NOT_AVAILABLE

{test=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)

```
[2018-05-15 23:46:40,132] WARN [Producer clientId=console-producer] Error while
fetching metadata with correlation id 14 : {test=LEADER_NOT_AVAILABLE} (org.apac
he.kafka.clients.NetworkClient)
[2018-05-15 23:46:40,266] WARN [Producer clientId=console-producer] Error while
fetching metadata with correlation id 15 : {test=LEADER_NOT_AVAILABLE} (org.apac
he.kafka.clients.NetworkClient)
^C[2018-05-15 23:46:40,394] WARN [Producer clientId=console-producer] Error whil
e fetching metadata with correlation id 16 : {test=LEADER_NOT_AVAILABLE} (org.ap
ache.kafka.clients.NetworkClient)
[root@tos opt]# {test=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkCl
ient)
bash: syntax error near unexpected token `org.apache.kafka.clients.NetworkClient'
```

Solutions: /opt/kafka/config/server.properties

Update the following information.

```
# it uses the value for "listeners" if configured. Otherwise, it will use the v
alue
# returned from java.net.InetAddress.getCanonicalHostName().
advertised.listeners=PLAINTEXT://localhost:9092
# Many listener names to security protocols, the default is for them to be the s
```

java.util.concurrent.ExecutionException:

org.apache.kafka.common.errors.TimeoutException: Expiring 1 record(s) for my-kafka-
topic-6: 30037 ms has passed since batch creation plus linger time

at

org.apache.kafka.clients.producer.internals.FutureRecordMetadata.valueOrError(FutureRe
cordMetadata.java:94)

at
org.apache.kafka.clients.producer.internals.FutureRecordMetadata.get(FutureRecordMetadata.java:64)

at
org.apache.kafka.clients.producer.internals.FutureRecordMetadata.get(FutureRecordMetadata.java:29)

at com.tos.kafka.MyKafkaProducer.runProducer(MyKafkaProducer.java:97)

at com.tos.kafka.MyKafkaProducer.main(MyKafkaProducer.java:18)

Caused by: org.apache.kafka.common.errors.TimeoutException: Expiring 1 record(s) for my-kafka-topic-6: 30037 ms has passed since batch creation plus linger time.

Solution:

Update the following in all the server properties: /opt/kafka/config/server.properties

```
# listeners = PLAINTEXT://your.host.name:9092
listeners=PLAINTEXT://tos.master.com:9093

# Hostname and port the broker will advertise to producers and consumers. If not
# set,
# it uses the value for "listeners" if configured. Otherwise, it will use the v
# value
# returned from java.net.InetAddress.getCanonicalHostName().
advertised.listeners=PLAINTEXT://tos.master.com:9093

# Maps listener names to security protocols, the default is for them to be the s
# ame. See the config documentation for more details
listener.security.protocol.map=PLAINTEXT:PLAINTEXT,SSL:SSL,SASL_PLAINTEXT:SASL_
PLAINTEXT,SASL_SSL:SASL_SSL
```

Its should be updated with your hostname and restart the broker

Changes in the following file, if the hostname is to be changed.

//kafka/ Server.properties and control center

/apps/confluent/etc/confluent-control-center/control-center-dev.properties

```
/apps/confluent/etc/ksql/ksql-server.properties  
/tmp/confluent.8A2Ii7O4/connect/connect.properties
```

Update localhost to resolve to the ip in /etc/hosts.

In case the hostname doesn't started, updated with ip address and restart the broker.

6. Annexure Code:

2. DumplogSegment

```
/opt/kafka/bin/kafka-run-class.sh kafka.tools.DumpLogSegments --deep-iteration --print-
data-log --files \
/tmp/kafka-logs/my-kafka-connect-o/00000000000000000000.log | head -n 4
```

```
[root@tos test-topic-0]# more 00000000000000000000.log
[root@tos test-topic-0]# cd ../
[root@tos kafka-logs]# cd my-kafka-connect-0/
[root@tos my-kafka-connect-0]# ls
00000000000000000000.index      0000000000000000000011.snapshot
00000000000000000000.log        leader-epoch-checkpoint
00000000000000000000.timeindex
[root@tos my-kafka-connect-0]# more *log
\██████████afka Connector.--More--(53%)

[root@tos my-kafka-connect-0]# pwd
/tmp/kafka-logs/my-kafka-connect-0
[root@tos my-kafka-connect-0]# /opt/kafka/bin/kafka-run-class.sh kafka.tools.Dum
pLogSegments --deep-iteration --print-data-log --files \
> /tmp/kafka-logs/my-kafka-connect-0/00000000000000000000.log | head -n 4
Dumping /tmp/kafka-logs/my-kafka-connect-0/00000000000000000000.log
Starting offset: 0
offset: 0 position: 0 CreateTime: 1530552634675 invalid: true keysize: -1 values
ize: 31 magic: 2 compresscodec: NONE producerId: -1 producerEpoch: -1 sequence:
-1 isTransactional: false headerKeys: [] payload: This Message is from Test File
.
offset: 1 position: 0 CreateTime: 1530552634677 invalid: true keysize: -1 values
ize: 43 magic: 2 compresscodec: NONE producerId: -1 producerEpoch: -1 sequence:
-1 isTransactional: false headerKeys: [] payload: It will be consumed by the Kaf
ka Connector.
[root@tos my-kafka-connect-0]#
```

3. Data Generator – JSON

Streaming Json Data Generator

Downloading the generator

You can always find the [most recent release](#) over on github where you can download the bundle file that contains the runnable application and example configurations. Head there now and download a release to get started!

Configuration

The generator runs a Simulation which you get to define. The Simulation can specify one or many Workflows that will be run as part of your Simulation. The Workflows then generates Events and these Events are then sent somewhere. You will also need to define Producers that are used to send the Events generated by your Workflows to some destination. These destinations could be a log file, or something more complicated like a Kafka Queue.

You define the configuration for the json-data-generator using two configuration files. The first is a Simulation Config. The Simulation Config defines the Workflows that should be run and different Producers that events should be sent to. The second is a Workflow configuration (of which you can have multiple). The Workflow defines the frequency of Events and Steps that the Workflow uses to generate the Events. It is the Workflow that defines the format and content of your Events as well.

For our example, we are going to pretend that we have a programmable [Jackie Chan](#) robot. We can command Jackie Chan through a programmable interface that happens to take json

as an input via a Kafka queue and you can command him to perform different fighting moves in different martial arts styles. A Jackie Chan command might look like this:

```
{
  "timestamp":"2015-05-20T22:05:44.789Z",
  "style":"DRUNKEN_BOXING",
  "action":"PUNCH",
  "weapon":"CHAIR",
  "target":"ARMS",
  "strength":8.3433
}
```

[view rawexampleJackieChanCommand.json](#) hosted with [by GitHub](#)

Now, we want to have some fun with our awesome Jackie Chan robot, so we are going to make him do random moves using our json-data-generator! First we need to define a Simulation Config and then a Workflow that Jackie will use.

SIMULATION CONFIG

Let's take a look at our example Simulation Config:

```
{
  "workflows": [{
    "workflowName": "jackieChan",
    "workflowFilename": "jackieChanWorkflow.json"
  }],
}
```

```
"producers": [{  
  "type": "kafka",  
  "broker.server": "192.168.59.103",  
  "broker.port": 9092,  
  "topic": "jackieChanCommand",  
  "flatten": false,  
  "sync": false  
}]  
}
```

[view rawjackieChanSimConfig.json](#) hosted with [by GitHub](#)

As you can see, there are two main parts to the Simulation Config. The Workflows name and list the workflow configurations you want to use. The Producers are where the Generator will send the events to. At the time of writing this, we have three supported Producers:

- A Logger that sends events to log files
- A [Kafka](#) Producer that will send events to your specified Kafka Broker
- A [Tranquility](#) Producer that will send events to a [Druid](#) cluster.

You can find the full configuration options for each on the [github](#) page. We used a Kafka producer because that is how you command our Jackie Chan robot.

WORKFLOW CONFIG

The Simulation Config above specifies that it will use a Workflow called jackieChanWorkflow.json. This is where the meat of your configuration would live. Let's take a look at the example Workflow config and see how we are going to control Jackie Chan:

```
{
  "eventFrequency": 400,
  "varyEventFrequency": true,
  "repeatWorkflow": true,
  "timeBetweenRepeat": 1500,
  "varyRepeatFrequency": true,
  "steps": [{
    "config": [{
      "timestamp": "now()",
      "style": "random('KUNG_FU','WUSHU','DRUNKEN_BOXING')",
      "action": "random('KICK','PUNCH','BLOCK','JUMP')",
      "weapon": "random('BROAD_SWORD','STAFF','CHAIR','ROPE')",
      "target": "random('HEAD','BODY','LEGS','ARMS')",
      "strength": "double(1.0,10.0)"
    }
  ],
  "duration": 0
}]
}
```

[view rawjackieChanWorkflow.json](#) hosted with [by GitHub](#)

The Workflow defines many things that are all defined on the github page, but here is a summary:

- At the top are the properties that define how often events should be generated and if / when this workflow should be repeated. So this is like saying we want Jackie Chan to do a martial arts move every 400 milliseconds (he's FAST!), then take a break for 1.5 seconds, and do another one.
- Next, are the Steps that this Workflow defines. Each Step has a config and a duration. The duration specifies how long to run this step. The config is where it gets interesting!

WORKFLOW STEP CONFIG

The Step Config is your specific definition of a json event. This can be any kind of json object you want. In our example, we want to generate a Jackie Chan command message that will be sent to his control unit via Kafka. So we define the command message in our config, and since we want this to be fun, we are going to randomly generate what kind of style, move, weapon, and target he will use.

You'll notice that the values for each of the object properties look a bit funny. These are special Functions that we have created that allow us to generate values for each of the properties. For instance, the “random('KICK','PUNCH','BLOCK','JUMP')” function will randomly choose one of the values and output it as the value of the “action” property in the command message. The “now()” function will output the current date in an ISO8601 date formatted string. The “double(1.0,10.0)” will generate a random double between 1 and 10

to determine the strength of the action that Jackie Chan will perform. If we wanted to, we could make Jackie Chan perform combo moves by defining a number of Steps that will be executed in order.

There are many more Functions available in the generator with everything from random string generation, counters, random number generation, dates, and even support for randomly generating arrays of data. We also support the ability to reference other randomly generated values. For more info, please check out the [full documentation](#) on the github page.

Once we have defined the Workflow, we can run it using the json-data-generator. To do this, do the following:

1. If you have not already, go ahead and [download the most recent release](#) of the json-data-generator.
2. Unpack the file you downloaded to a directory.

```
(tar -xvf json-data-generator-1.4.0-bin.tar -C /apps )
```

3. Copy your custom configs into the conf directory
4. Then run the generator like so:
 1. `java -jar json-data-generator-1.4.0.jar conf/jackieChanSimConfig.json`

You will see logging in your console showing the events as they are being generated. The jackieChanSimConfig.json generates events like these:

```
{"timestamp":"2015-05-20T22:21:18.036Z","style":"WUSHU","action":"BLOCK","weapon":"CHAIR","target":"BODY","strength":4.7912}
{"timestamp":"2015-05-20T22:21:19.247Z","style":"DRUNKEN_BOXING","action":"PUNCH","weapon":"BROAD_SWORD","target":"ARMS","strength":3.0248}
{"timestamp":"2015-05-20T22:21:20.947Z","style":"DRUNKEN_BOXING","action":"BLOCK","weapon":"ROPE","target":"HEAD","strength":6.7571}
{"timestamp":"2015-05-20T22:21:22.715Z","style":"WUSHU","action":"KICK","weapon":"BROAD_SWORD","target":"ARMS","strength":9.2062}
{"timestamp":"2015-05-20T22:21:23.852Z","style":"KUNG_FU","action":"PUNCH","weapon":"BROAD_SWORD","target":"HEAD","strength":4.6202}
{"timestamp":"2015-05-20T22:21:25.195Z","style":"KUNG_FU","action":"JUMP","weapon":"ROPE","target":"ARMS","strength":7.5303}
{"timestamp":"2015-05-20T22:21:26.492Z","style":"DRUNKEN_BOXING","action":"PUNCH","weapon":"STAFF","target":"HEAD","strength":1.1247}
{"timestamp":"2015-05-20T22:21:28.042Z","style":"WUSHU","action":"BLOCK","weapon":"STAFF","target":"ARMS","strength":5.5976}
```

```
{"timestamp":"2015-05-20T22:21:29.422Z","style":"KUNG_FU","action":"BLOCK","weapon":"ROPE","target":"ARMS","strength":2.152}  
{"timestamp":"2015-05-20T22:21:30.782Z","style":"DRUNKEN_BOXING","action":"BLOCK","weapon":"STAFF","target":"ARMS","strength":6.2686}  
{"timestamp":"2015-05-20T22:21:32.128Z","style":"KUNG_FU","action":"KICK","weapon":"BROAD_SWORD","target":"BODY","strength":2.3534}
```

[view rawjackieChanCommands.json](#) hosted with [by GitHub](#)

If you specified to repeat your Workflow, then the generator will continue to output events and send them to your Producer simulating a real world client, or in our case, continue to make Jackie Chan show off his awesome skills. If you also had a Chuck Norris robot, you could add another Workflow config to your Simulation and have the two robots fight it out! Just another example of how you can use the generator to simulate real world situations.

7. Pom.xml (Standalone)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.hp.tos</groupId>
  <artifactId>LearningKafkaZK</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <description>All About Kafka</description>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <algebird.version>0.13.4</algebird.version>
    <avro.version>1.11.3</avro.version>
    <confluent.version>7.9.0</confluent.version>
    <confluent.version.latest>8.2.0</confluent.version.latest>
    <kafka.version>3.9.0</kafka.version>
    <kafka.scala.version>2.13</kafka.scala.version>
  </properties>
  <repositories>
    <repository>
      <id>confluent</id>
```

```

        <url>https://packages.confluent.io/maven/</url>
    </repository>
</repositories>

<pluginRepositories>
    <pluginRepository>
        <id>confluent</id>
        <url>https://packages.confluent.io/maven/</url>
    </pluginRepository>
</pluginRepositories>
<dependencies>
    <dependency>
        <groupId>io.confluent</groupId>
        <artifactId>kafka-streams-avro-serde</artifactId>
        <version>${confluent.version}</version>
    </dependency>
    <!-- Apache Avro Core -->
    <dependency>
        <groupId>org.apache.avro</groupId>
        <artifactId>avro</artifactId>
        <version>${avro.version}</version>
    </dependency>
    <dependency>
        <groupId>io.confluent</groupId>
        <artifactId>kafka-avro-serializer</artifactId>

```

```
    <version>${confluent.version}</version>
</dependency>
<dependency>
    <groupId>io.confluent</groupId>
    <artifactId>kafka-schema-registry-client</artifactId>
    <version>${confluent.version}</version>
</dependency>
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>${kafka.version}</version>
</dependency>
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams</artifactId>
    <version>${kafka.version}</version>
</dependency>

<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams-test-utils</artifactId>
    <version>${kafka.version}</version>
    <scope>test</scope>
</dependency>
```

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.6.2</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-core</artifactId>
  <version>1.2.6</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.6</version>
</dependency>
</dependencies>

<build>
  <plugins>

    <plugin>
      <groupId>org.apache.avro</groupId>
      <artifactId>avro-maven-plugin</artifactId>
      <version>${avro.version}</version>
```

```

        <executions>
            <execution>
                <phase>generate-sources</phase>
                <goals>
                    <goal>schema</goal>
                </goals>
                <configuration>
                    <sourceDirectory>
                        ${project.basedir}/src/main/resources/avro</sourceDirectory>
                    <includes>
                        <include>*.avsc</include>
                    </includes>
                    <outputDirectory>${project.basedir}/src/main/java</outputDirectory>
                </configuration>
            </execution>
        </executions>
    </plugin>
</plugins>
</build>

</project>

```

Resources

<https://developer.ibm.com/hadoop/2017/04/10/kafka-security-mechanism-saslplain/>
<https://sharebigdata.wordpress.com/2018/01/21/implementing-sasl-plain/>
<https://developer.ibm.com/code/howtos/kafka-authn-authz>
<https://github.com/confluentinc/kafka-streams-examples/tree/4.1.x/>
<https://github.com/spring-cloud/spring-cloud-stream-samples/blob/master/kafka-streams-samples/kafka-streams-table-join/src/main/java/kafka/streams/table/join/KafkaStreamsTableJoin.java>
<https://docs.confluent.io/current/ksql/docs/tutorials/examples.html#ksql-examples>